Chapitre 0 — Avant tout | Partie D — Des formes | leçon 0.61.2 sur 153 | difficulté 1 / 10

0.61.2. Aller et revenir en diagonale — doublé, deux positions

La forme du A autour de (10, 8), puis celle du B autour de (4, 4).

1. Ce qu'il faut comprendre

C'est le 0.61 et le 0.61.1 réunis : la même forme à deux places, l'une après l'autre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <aller_retour>` → 0.59.3. La fonction aller_retour() — aller et revenir
- `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 **Les mots de cette leçon — si tu débutes**

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <aller_retour>  // une tuile qui va et revient`	Demande à la console « aller_retour » (ce que c'est : une tuile qui va et revient). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui va et revient ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`int main() {  // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`// aller_retour(x, y, tuile, pasX, pasY, vitesse, fois) : de (x, y) à`	Un commentaire : une phrase pour celui qui lit (« aller_retour(x, y, tuile, pasX, pasY, vitesse, fois) : de (x, y) à »). La console l'ignore.
`// (x + pasX, y + pasY), puis retour. pasX et pasY non nuls → en diagonale.`	Un commentaire : une phrase pour celui qui lit (« (x + pasX, y + pasY), puis retour. pasX et pasY non nuls → en diagonale. »). La console l'ignore.
`// 1. Le A, au milieu`	Un commentaire : une phrase pour celui qui lit (« 1. Le A, au milieu »). La console l'ignore.
`aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2); // tout droit, jusqu'à (15, 8)`	Appel de la fonction aller_retour de la console, avec les arguments 10, 8, ALPHABET[0], 5, 0, 250, 2. Ce qu'elle fait : va jusqu'à (x + pasX, y + pasY) et revient, « fois » fois : 5, 0 tout droit à droite ; 4, 4 en diagonale vers le bas à droite. Finit à sa place de départ. (Le commentaire le dit : « tout droit, jusqu'à (15, 8) ».)
`aller_retour(10, 8, ALPHABET[0], 4, 4, 100, 1); // en diagonale, jusqu'à (14, 12)`	Appel de la fonction aller_retour de la console, avec les arguments 10, 8, ALPHABET[0], 4, 4, 100, 1. Ce qu'elle fait : va jusqu'à (x + pasX, y + pasY) et revient, « fois » fois : 5, 0 tout droit à droite ; 4, 4 en diagonale vers le bas à droite. Finit à sa place de départ. (Le commentaire le dit : « en diagonale, jusqu'à (14, 12) ».)
`// 2. Le B, en haut à gauche`	Un commentaire : une phrase pour celui qui lit (« 2. Le B, en haut à gauche »). La console l'ignore.
`aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2); // tout droit, jusqu'à (9, 4)`	Appel de la fonction aller_retour de la console, avec les arguments 4, 4, ALPHABET[1], 5, 0, 250, 2. Ce qu'elle fait : va jusqu'à (x + pasX, y + pasY) et revient, « fois » fois : 5, 0 tout droit à droite ; 4, 4 en diagonale vers le bas à droite. Finit à sa place de départ. (Le commentaire le dit : « tout droit, jusqu'à (9, 4) ».)
`aller_retour(4, 4, ALPHABET[1], 4, 4, 100, 1); // en diagonale, jusqu'à (8, 8)`	Appel de la fonction aller_retour de la console, avec les arguments 4, 4, ALPHABET[1], 4, 4, 100, 1. Ce qu'elle fait : va jusqu'à (x + pasX, y + pasY) et revient, « fois » fois : 5, 0 tout droit à droite ; 4, 4 en diagonale vers le bas à droite. Finit à sa place de départ. (Le commentaire le dit : « en diagonale, jusqu'à (8, 8) ».)
`while (true) {  // La boucle de jeu, pour toujours :`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « La boucle de jeu, pour toujours : ».)
`image();  //  attendre l'image suivante.`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attendre l'image suivante. ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1400 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  aller_retour()  400 o  ← main()  — une fonction fournie par le langage
  main()          129 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()        × 3    39 o sur place, dans aller_retour()  + AdresseCase (10 o)
  effacer()      × 2    30 o sur place, dans aller_retour()  + AdresseCase (10 o), EffacerCases (12 o)
  image()        × 3     9 o sur place, dans aller_retour(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  aller_retour() × 4   122 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <aller_retour>   // une tuile qui va et revient
#include <ALPHABET>       // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

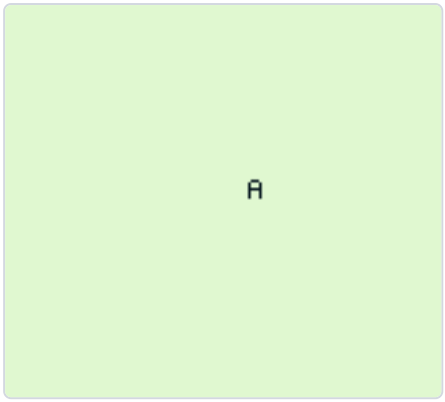
```
int main() {          // Le programme commence ici.
  // aller_retour(x, y, tuile, pasX, pasY, vitesse, fois) : de (x, y) à
  // (x + pasX, y + pasY), puis retour. pasX et pasY non nuls → en diagonale.
  // 1. Le A, au milieu
  aller_retour(10, 8, ALPHABET[0], 5, 0, 250, 2);   // tout droit, jusqu'à (15, 8)
  aller_retour(10, 8, ALPHABET[0], 4, 4, 100, 1);   // en diagonale, jusqu'à (14, 12)
  // 2. Le B, en haut à gauche
  aller_retour(4, 4, ALPHABET[1], 5, 0, 250, 2);    // tout droit, jusqu'à (9, 4)
  aller_retour(4, 4, ALPHABET[1], 4, 4, 100, 1);    // en diagonale, jusqu'à (8, 8)

  while (true) {      // La boucle de jeu, pour toujours :
    image();          //   attendre l'image suivante.
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

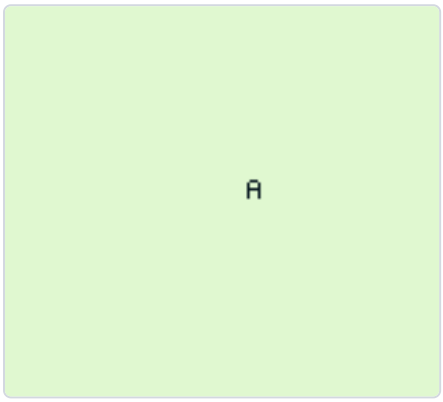
la toute première image — la console vient de démarrer — image 1



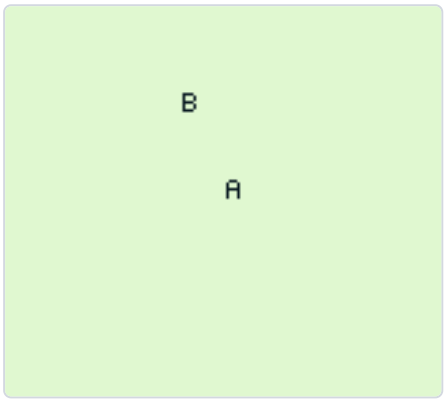
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> La forme du A au milieu, puis celle du B en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (10, 8) (10,8)

✓ le B finit en (4, 4) (4,4)

Ensuite : **0.62. Deux formes ensemble** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++